



DURBAN UNIVERSITY OF TECHNOLOGY
INYUVESI YASETHEKWINI YEZOBUCHWEPHESHE

FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

2024

MID YEAR SUPPLEMENTARY EXAMINATION

MARKING MEMORANDUM

INSTRUCTIONAL PROGRAMME : B INFORMATION & COMMUNICATION TECHNOLOGY

INSTRUCTIONAL OFFERING : INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

PAPER NUMBER : 1

SUBJECT CODE/S : IPRT301

DATE :

DURATION : 3 HOURS

TIME : 09H00 - 12H00

TOTAL MARKS : 100

NUMBER OF PAGES : 11 (INCLUDING COVER PAGE)

EXAMINER/S : BOAMAH-ABU, CHARLES (PhD)

MODERATOR/S : IBIDUN OBAGBUWA, CHRISTIANA (PhD)

INSTRUCTIONS/REQUIREMENTS:-

1. This examination is closed book
2. Answer all questions
3. Ensure your handwriting is readable

Do not turn the page until permission is given

Question 1

6 marks

In each of the following write **T** (for **true**) or **F** (for **false**) for your answer.

- 1 Distributed applications must be tightly coupled
- 2 An abstract class cannot contain method definition
- 3 An abstract class can be instantiated
- 4 Method overload occurs in the same class
- 5 In polymorphism, an appropriate behaviour of an object is determined at run time
- 6 The adapter pattern applies if only one instance of a class is to be created

Answer: 1. F 2. F 3. F 4.T 5. T 6. F

Question 2

9 marks

Briefly answer the following.

1. Explain how Integrative programming enables efficient use of organizational computing resources? (3 marks)
2. Give one reason why XML is important in distributed applications. (2 marks)
3. Give one benefit of design patterns. (2 marks)
4. Give one reason why Microservice architecture is preferable to Webservice. (2 marks)

Answers

1. Any correct answer which emphasizes efficiency (cost, time, etc.), e.g., Integrative programming allows heterogeneous resources (data, hardware and software) to seamlessly work together; e.g., different software and hardware share the same data (instead of formatting for different platforms) ✓✓. This reduces cost✓.
2. Any correct answer, e.g., XML allows different applications to share data ✓✓
3. Any correct answer, e.g., Design patterns provide a template for solving common problems in software development ✓✓
4. Any correct answer, e.g., Microservice architecture supports small scale, independent applications. This reduces maintenance cost. ✓✓

Question 3

15 marks

1. **X, Y and Z** are interfaces. State whether the following is valid or invalid about interfaces. Explain your answer. **(8 marks)**

1.1. `X x1=new X1();`

1.2. `public interface Y implements X{`

`...`

`...`

`}`

1.3. `public interface Y extends X{`

`...`

`...`

`}`

1.4. `public interface Z, Y extends X{`

`...`

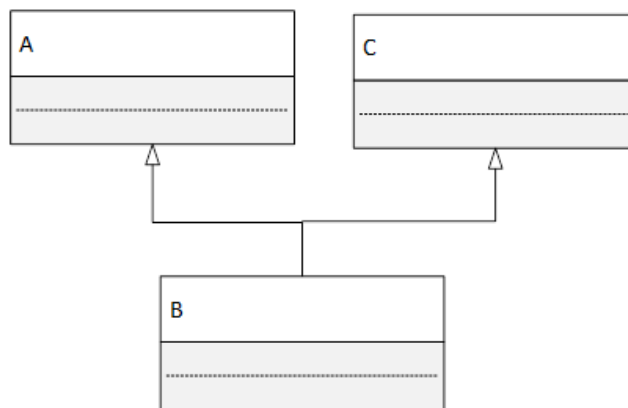
`...`

`}`

2. The UML diagram below shows class **B** inherits classes **A** and **C**.

2.1. Explain why the diagram cannot be converted to a Java program? **(3 marks)**

2.2. If **A** contains only constants and abstract methods and **B** contains instance variables and method definitions, explain how you can resolve the problem. **(4 marks)**



Answers:

1.1. Invalid: cannot create an instance of an interface✓✓

1.2. Invalid: X cannot implement in Y because an interface cannot contain method✓✓

1.3.Valid: an interface can inherit another interface✓✓

1.4.Valid: an interface can extend multiple interfaces (note: ',' is the correct separator) ✓✓

2.1.The diagram contains multiple inheritance; ✓ Java does not allow multiple inheritance✓✓

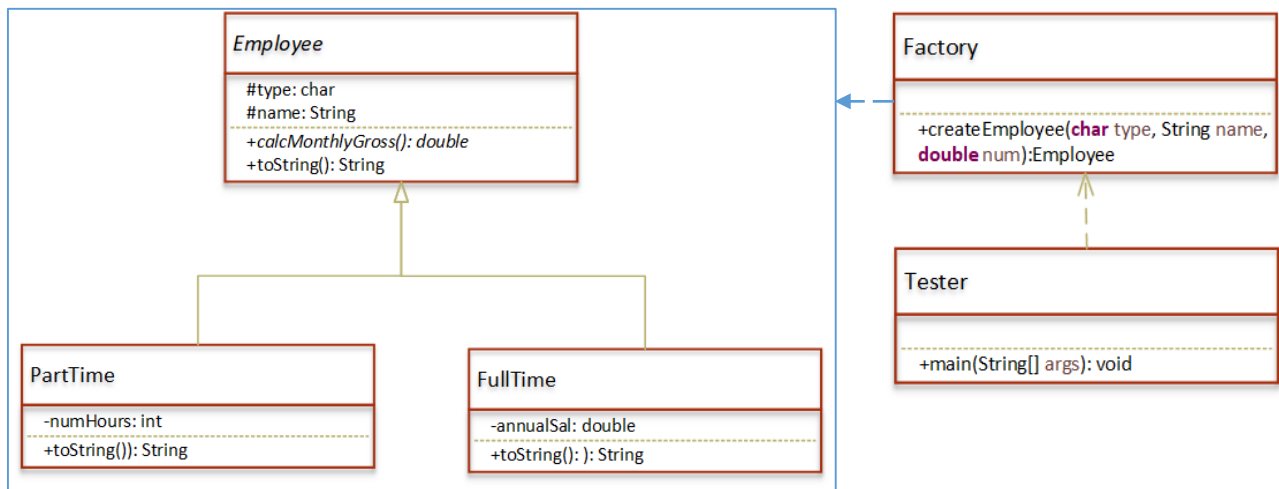
2.2.Convert one of the classes to an interface (A contains only constants and abstract methods and, therefore, can be converted to an interface) ✓✓. Let B implement A and inherit C.

✓✓ This removes the multiple inheritance

Question 4

20 marks

The UML diagram below shows the classes **Employee**, **PartTime**, **FullTime**, **Factory** and **Tester**.



- Employee** is an abstract class and contains **type** and **name** (of an employee); the abstract method **calcMonthlyGross()** returns the monthly gross income of an employee. **PartTime** and **FullTime** inherit **Employee**. The class contains appropriate **constructors** and **toString()** method
- PartTime** contains **numOfHours** (the number of hours an employee worked in a month) and overrides **calcMonthlyGross()** to return the monthly gross income as R80 per hour. The class contains appropriate **constructors** and **toString()** method
- FullTime** contains **annualSal** (the annual gross income of an employee) and overrides **calcMonthlyGross()** to return the monthly gross income as 1/12 of the annual gross income. The class contains appropriate **constructors** and **toString()** method
- Factory** is a factory class. **createEmployee(char type, String name, double num)** takes **type** and **name** (of an employee) and a **number** as parameters. If **type** is:
 - ‘P’: the method uses the parameters to create and return an instance of **PartTime**
 - ‘F’: the method uses the parameters to create and return an instance of **FullTime**
- Tester** creates an instance of **Factory** and calls its method to return an instance of a subclass (i.e., **PartTime** or **FullTime**) of **Employee**. The subclass is then displayed on the screen.

Use the information above to answer Questions 1 to 5 below.

- Write the method **calcMonthlyGross()** in the class **Employee**. (2 marks)

Answer: **public abstract double calcMonthlyGross();**✓✓

- Write the header of the class **PartTime**. (2 marks)

Answer: **public class PartTime extends Employee**✓✓

- Write the method **calcMonthlyGross()** in the class **PartTime**. (3 marks)

Answer: **public double calcMonthlyGross() {** ✓ **return** ✓ **numOfHours*RATE;** ✓
✓}

4. Below, complete the method **createEmployee**(char type, String name, double num) in the class **Factory**. Assume the relevant classes and their methods (including **appropriate constructors**) have been implemented. (8 marks)

```
_____createEmployee(char type, String name, double num) {  
_____  
_____  
_____  
_____  
_____  
}
```

Answer

```
public Employee ✓createEmployee(char type, String name, double num) {  
    ✓if(type=='P') {  
        ✓✓new PartTime(type, name, num);  
    }  
    ✓if(type=='F') {  
        ✓✓return new FullTime(type, name, num);  
    }  
    ✓else return null;  
}
```

5. Assume the relevant classes and their methods (including **appropriate constructors** and **toString()** methods) have been implemented. Below, complete **main()** in **Tester** as follows:
1. Create an instance of **Factory** called **f**. (1 mark)
 2. Use **f** to call an appropriate method (with the values 'P', "Kofi Mensah", 120.00) to return the instance of a subclass. (3 marks)
 3. Display the instance of the subclass on the screen. (1 mark)

```
...  
...  
public class Tester {  
    public static void main(String[] args) {  
_____  
_____  
_____  
_____  
    }  
}
```

Answer

```
public static void main(String[] args) {  
    ✓Factory f=new Factory();  
    ✓✓✓Employee e2=f.createEmployee('P', "Kofi Mensah", 120.00);  
    ✓System.out.println(e2);  
}
```

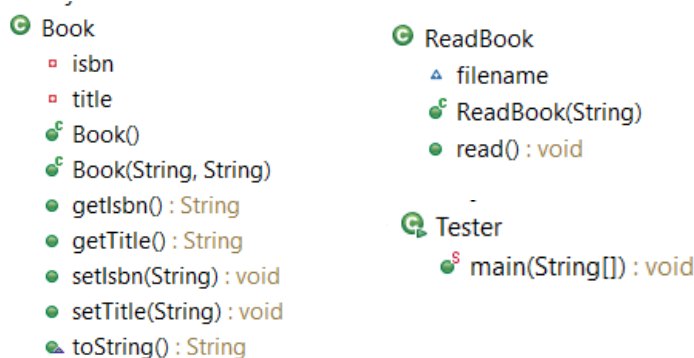
Question 5

23 marks

The XML below is in the file **library.xml**.

```
<?xml version="1.0" encoding="UTF-8"?>
<Library>
  <book isbn="001-002-002-1">
    <title>Java for dummies</title>
  </book>
  <book isbn="001-002-002-2">
    <title>Design patterns</title>
  </book>
  <book isbn="001-002-002-3">
    <title>Data mapping and exchange</title>
  </book>
</Library>
```

An application to read the XML in **library.xml** consists of the classes **Book**, **ReadBook** and **Tester** (below).



ReadBook reads the XML to create **Book** objects. The instance variable **filename** is the name of a file. The constructor uses the parameter to initialize **filename**. The method **read()** uses **filename** to set up an **XMLStreamReader** (called **xr**) to read XML from **library.xml**.

Assume **Book** has been implemented. Below (see **next page**) are parts of the method **read()** in **ReadBook**.

1. Use the information above to answer **Questions 1.1 to 1.4**.
 - 1.1. Complete the code segment (**marked 1.1**) to open the file. (2 marks)
 - 1.2. Complete the code segment (**marked 1.2**) for a loop to read until end of the XML document. (2 marks)
 - 1.3. Complete the code segment (**marked 1.3**) as follows. If the variable **event** is the start element **<book>**, get the value of the **isbn** attribute; create an instance of **Book** and assign to **bk**; and use the value of the attribute to set the **isbn** of **bk**. (7 marks)
 - 1.4. Complete the code segment (**marked 1.4**) as follows. If the variable **event** is text, get the text and use to set the **title** of **bk**. (4 marks)

```

public void read(){
    try {
        //1.1. Complete the code below
        FileReader fr= _____;
        XMLStreamReader xr=null;//
        XMLInputFactory xmlfactory=XMLInputFactory.newInstance();
        xr=xmlfactory.createXMLStreamReader(fr);
        Book bk=null;
        int event;
        //1.2. Complete the code below
        while(_____){
            event = xr.next();
            switch(event) {
                //1.3. Complete the code below
                case _____:{
                    _____
                    _____
                    _____
                }
                break

                //1.4. Complete the code below
                case _____:{
                    _____
                    _____
                    _____
                }
                break
                ...
                ...
            }
        }
    }
    ...
}

```

Answer

```

//Complete the code for Question 1.1
FileReader fr=new FileReader(filename); ✓✓
//Complete the code for Question 1.2
while(xr.hasNext())✓✓
//Complete the code for Question 1.3
case XMLStreamConstants.START_ELEMENT: ✓✓{
    if(xr.getLocalName().equals("book"))✓✓ {
        book=new Book();✓
        book.setCode(xr.getAttributeValue(0)) ✓✓;
    }
}

```

break;

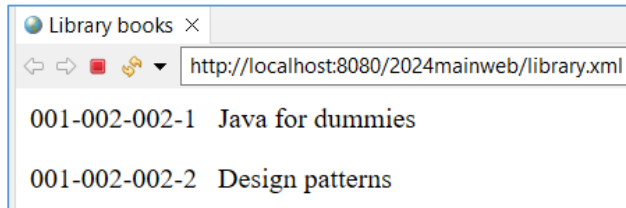
//Complete the code for Question 1.4

case XMLStreamConstants.CHARACTERS: ✓✓{

book.setTitle(xr.getText());✓✓

}

2. **library.xsl** contains the XSL code to format the XML in **library.xml** to display in a web page as shown below.



2.1. Complete the code for **library.xsl**.

(6 marks)

```
...
...
<xsl:template match="/ Library ">
    <xsl:for-each select="_____">
        <_____="_____" /> &#160;
        <_____="_____" /> <p/>
    </xsl:for-each>
</xsl:template>
...
...
```

Answer:

```
<xsl:template match="/">
    <xsl:for-each select="Library/book"✓✓>
        <xsl:value-of select="@isbn"✓ /> &#160;
        <xsl:value-of select="title"✓ /> <p/>
    </xsl:for-each>
</xsl:template>
```

2.2. Complete the code below in **library.xml** to link to **library.xsl**.

(2 marks)

```
<?xml-stylesheet type="_____" href="_____"?>
```

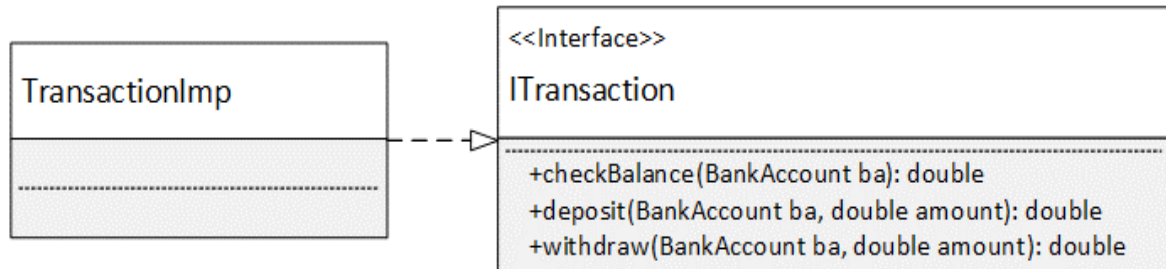
Answer

```
<?xml-stylesheet type="text/xsl✓" href="library.xsl✓"?>
```

Question 6

27 marks

An RMI application allows transactions (e.g., check balance, deposit amount and withdraw amount) on a bank account. The UML diagram below shows the remote interface and remote class.



- **checkBalance**(BankAccount ba) takes a BankAccount as parameter and returns the account balance
- **deposit**(BankAccount ba, double amount) takes a BankAccount and the amount to deposit as parameters, adds the amount to the account balance and returns the updated balance
- **withdraw**(BankAccount ba, double amount) takes a BankAccount and the amount to withdraw as parameters, deducts the amount from the account balance and returns the updated balance

The class **BankAccount** (below) contains **accNum** (the account number), **balance** (the account balance) and **name** (customer name).

```

BankAccount
  ▢ accNum
  ▢ balance
  ▢ custName
  ● BankAccount(String, String, double)
  ● getAccNum() : String
  ● getBalance() : double
  ● getCustName() : String
  ● setAccNum(String) : void
  ● setBalance(double) : void
  ● setCustName(String) : void
  ● toString() : String
    
```

Use the information to answer **Questions 1 to 6** below.

1. Write the class header of **BankAccount** for this distributed application. (2 marks)

Answer

```
public class BankAccount implements Serializable✓✓
```

2. Write the code for **ITransaction**. (8 marks)

Answer

```
public interface ITransaction extends Remote✓✓{
    public abstract double checkBalance(BankAccount ba) throws Exception; ✓✓
}
```

```

public abstract double deposit(BankAccount ba, double amount) throws Exception; ✓✓
public abstract double withdraw(BankAccount ba, double amount) throws Exception; ✓✓
}

```

3. Write the class header for **TransactionImp**. (3 marks)

Answer

```

public class TransactionImp ✓ extends UnicastRemoteObject ✓ implements
ITransaction ✓

```

4. Write the method **deposit**(BankAccount ba, **double** amount) in **TransactionImp**. (4 marks)

Answer

```

public double deposit(BankAccount ba, double amount) throws Exception ✓ {
    ✓ ba.setBalance(ba.getBalance()+amount); ✓
    return ba.getBalance(); ✓
}

```

5. Complete the code segment below for a class **Server** which creates, registers and binds an instance of **TransactionImp** (called **ti**) with the reference "account". Assume **TransactionImp** has been implemented. Use port **2001**. (5 marks)

```

...
...
public class Server {
    public static void main(String[] args) throws Exception{
        TransactionImp ti=_____
        Registry reg=_____
        reg.rebind(_____);
    }
}

```

Answer

```

...
...
public class Server {
    public static void main(String[] args) throws Exception{
        TransactionImp ti=new TransactionImp(); ✓
        Registry reg=LocateRegistry.createRegistry(2001); ✓✓
        reg.rebind("account", ti); ✓✓
    }
}

```

6. Complete the code segment below for a class **Client** which creates an instance of the remote object (called **it**) and calls a method to return the balance of a BankAccount (called

b). Assume the relevant classes and their methods have been implemented. Use the local host and port number 2001. (5 marks)

```
...  
...  
public class Client {  
    public static void main(String[] args) throws Exception{  
        BankAccount b=new BankAccount("400-000-000-1","Kofi Manu", 1000);  
        ITransaction it=_____  
        System.out.println("Balance=R "+_____) );  
    }  
}
```

Answer

```
...  
...  
public class Client {  
    public static void main(String[] args) throws Exception{  
        BankAccount b=new BankAccount("400-000-000-1","Kofi Manu", 1000);  
        ITransaction it=(ITransaction) ✓Naming.lookup✓ ("rmi://localhost:2001/account");✓  
        System.out.println("Balance=R "+it.checkBalance(b) ✓✓);  
    }  
}
```